NEAT-Based Neural Controller for Robust Line-Following Robots

Yazan AlAtout[1] and Raneem Qaddoura[2]

[1]Department of Computer Science, Al Hussein Technical University, Amman, 11831, Amman Governorate, Jordan.

Contributing authors: 23110209@htu.edu.jo; Raneem.Qaddoura@htu.edu.jo;

Abstract

This paper explores the question of whether a NeuroEvolution of Augmenting Topologies (NEAT)-based control system can outperform a traditional Proportional-Integral-Derivative (PID) controller for a line-following robot. Traditional PID controllers, while widely used, suffer from limitations such as difficult manual tuning, poor adaptability to dynamic environments, and an inability to handle sharp turns. In contrast, this study proposes using NEAT, a genetic algorithm that evolves the topology and weights of neural networks, to enable the robot to autonomously learn optimal control behaviors. The study developed a 2D simulation using PyGame to train and test the NEAT algorithm, comparing its performance against an optimized PID controller. Results show a critical trade-off between speed and precision. While the PID controller demonstrated superior performance in terms of high-speed lap completion with an average time of 20.93 seconds, the NEAT controller struggled at speeds above 300 px/s, with its fastest completed lap being 48.20 seconds. However, the NEAT controller excelled in maintaining a precise trajectory, achieving a significantly lower average Root Mean Squared Error (RMSE) of 0.916 in line deviation, compared to the PID's average RMSE of 1.99. This indicates that neuroevolution offers a promising path for developing more sophisticated and robust robotic controllers that can overcome the limitations of manual tuning and static parameters, particularly in applications where precision is more critical than speed.

Keywords: Robot, PID, NEAT, AI, line-follower, NeuroEvolution, Neural Networks

1 Introduction

In the field of robotics, AI has seen a huge development in algorithms used to control robots, such as using a Q-learning controller in line-following robots [1] or employing deep reinforcement learning for robotics in wheeled navigation, legged navigation, and aerial navigation [2]. Artificial neural networks (ANN) have become a wide field of research in robotics; moreover, NeuroEvolution has been shown to be a worthy competitor of reinforcement learning in developing and optimizing ANNs to control robots [3]. Line-following robots are designed to detect and follow a generally predefined path. These systems have proven valuable in warehouse automation to move goods from one place to another [4]. Among the many techniques developed to control line-following robots, the PID controller stands out. The PID controller is the most popular controller for controlling line-following robots, and some studies have shown that NEAT can be used to evolve ANNs for controlling autonomous vehicles, offering promising avenues to develop more sophisticated and adaptable robotic systems [5].
Traditional line following robots use a PID controller to control the stability of the robot on the line [6]. However, the PID controller has several limitations. Firstly, difficult manual tuning of the three constants (K_p, K_i, and K_d), which is mostly achieved using a heuristic approach for line following robots [7]. Secondly, PID controllers by themselves "could not handle sharp 45 degree turns" [7], which highlights the need for a robust method to manage such environments. Thirdly, PID controllers' goal is to minimize line tracking error, not global efficiency, thus PID does not necessarily find the shortest path [6]. Lastly, Poor Adaptability: changes in the motor speed or external disturbances will likely result in the PID controller failing, because of the fixed parameters [6].
This paper explores the question of whether a NEAT-based control system can outperform a PID controller by minimizing errors and reducing lap time for a line-following robot. To investigate this, the study proposes a new method for implementing line-following robots by using an algorithm called NeuroEvolution of Augmenting Topologies (NEAT), a genetic algorithm developed to generate and evolve neural networks and optimize their weights and size. In contrast to PID, which relies on predefined formulas, NEAT allows the robot to learn optimal behaviors through evolutionary training, making it more adaptable to different environments. This study aims to develop a NEAT-based control system for a line-following robot and compare its performance with traditional PID controllers in terms of accuracy, adaptability, and lap completion time.
The remainder of this paper is structured as follows: Section 2 provides a review of related work, discussing existing approaches for line-following robots, including PID controllers and various neural network implementations. Section 3 details the NeuroEvolution of Augmenting Topologies (NEAT) algorithm and its implementation in our simulation. Section 4 describes the data collected from both the NEAT and PID controllers. Section 5 outlines the research methodology, including the design of the 2D simulation, the robot, and the track. Finally, Section 6 presents a direct comparison and discussion of the results, followed by the conclusion of our findings and future work.

2 Related Work

Recent advances in autonomous robotics have explored different control methods in line-following robots, including classic PID controllers and those based on neural networks. This section reviews important works such as optimized PID controllers, use of neural networks, and evolutionary algorithms in robotic control, with their performance, limitations, and importance in generalization in dynamic environments.

The PID controller is the most popular controller for line-following robots, many researchers have discussed their implementation of it in their line following robots. Öğüten and Kabas [7] developed an inexpensive PID controller with heuristic tuning supported by an open-loop technique tailored for sharp turn navigation. Their improved PID implementation reduced lap time from 9.2 seconds to 7.8 seconds but required manual adjustments with limited generalizability outside of lab-tested track configurations. The work highlighted the sensitivity of PID controllers to noise in their environment (e.g., dust) as well as in spacing of sensors, motivating quick fix such as conditional turn-specific logic. While successful under laboratory circumstances, this method lacked sufficient adaptability in case of changing line width or dynamic perturbations. Balaji et al. [6] proposed an adaptive PID controller for high-speed line tracking, combining limited integral action (summing only the last 10 error values) and dynamic speed reduction to handle curvature changes. Their robot achieved 120 cm/s on complex tracks, outperforming classical PID (90 cm/s). However, the system relied on IR sensors prone to ambient light interference, and tuning was empirical, lacking theoretical robustness. The study highlighted the trade-off between speed and stability in sharp turns, which required open-loop saturation handling, which is a limitation for fully autonomous operation.

Other autonomous robot implementations used neural network-based controllers. Leal et al. [8] did a comparison of PID, LSTM, and CNN controllers in line-following robots. The results showed that the PID controller outperformed in predetermined tracks with a mean absolute error (MAE) of 2257 at an 80 Hz sampling rate, thereby outperforming LSTM (MAE: 3435) and CNN (MAE: 3857). However, the PID controller also suffered from sharp turns and required manual tuning, whereas neural network models showed an ability to learn complex routes but suffered from problems with computational delay (sampling rates: 24–25 Hz). A main limitation recognized involved the use of the PID controller to gather data for generating the data for training LSTM and CNN models, which limited their ability to learn behaviors that would surpass the performance of the PID itself. Lauckner and Kolivand [5] applied the NEAT algorithm to autonomous vehicles, demonstrating its potential to create neural networks that improve adaptability in dynamic road situations. Their simulation results showed that NEAT can improve safety by enabling vehicles to change their behavior in real-time through generational evolution. The study showed that there are limitations in terms of scalability for complex environments and the high computational demands. Although the focus was autonomous vehicles, their results show the broader applicability of NEAT in robotic systems that require adaptive decision-making. Minaya et al. (2024) compared a multilayer neural network to PID and fuzzy logic in line-following robots, finding the neural network corrected errors about 0.1 seconds faster than traditional controllers [9]. Broader surveys concur that learning-based controllers can

capture task-specific structure that classical tuning misses, albeit with higher sample and compute demands [10, 11]. Beyond NEAT's core complexification [12], real-time NEAT (rtNEAT) demonstrated online adaptation in interactive environments [13], while HyperNEAT scaled the approach to large, geometry-aware networks [14].
Reinforcement learning approaches were also tested in line following robots. R. J. Alitappeh [15] presents a dual-branch convolutional neural network (CNN) architecture that simultaneously handles line tracking via infrared sensors and obstacle avoidance using ultrasonic inputs. Trained end-to-end on both simulated and real-world datasets, with reinforcement learning fine-tuning, the system achieved 92% accuracy on complex line paths and 85% success in obstacle avoidance. It surpassed traditional PID controllers, especially under variable lighting and dynamic obstacles, and ran in real time on a Jetson Nano at 20 Hz. However, its dependence on GPU hardware, extensive simulation training requirements, and occasional sensor fusion misalignments limit its scalability and robustness. While it advances deep learning navigation, its static design contrasts with evolutionary approaches like NEAT, which could adaptively optimize sensor integration and reduce training complexity. Saadatmand et al [1] proposed a novel Simulated Annealing (SA)-based Q-learning controller for line-following robots, overcoming limitations of conventional PID controllers and standard Q-learning. Their methodology includes a full mathematical model of robot motion and applies SA to adaptively reduce exploration in Q-learning as training progresses. This method outperformed traditional proportional control and ε-greedy Q-learning in both simulation and real-world tests. The MIMO SA-Q learning controller gave the best performance in complex paths, demonstrating superior path tracking and speed. However, the technique requires extensive training and its performance can degrade under real-world noise and mechanical uncertainties, indicating sensitivity to hardware inconsistencies and longer training times for high-control-resolution models.
The existing literature on line-following robots concentrates on PID or neural networks (LSTM/CNN) [7, 8], neglecting evolutionary strategies such as NeuroEvolution of Augmenting Topologies (NEAT). PID controllers face inherent limitations in generalization due to their dependence on manual tuning and static parameters [6, 7]. Lauckner and Kolivand [5] demonstrated the potential of NEAT in autonomous vehicles, but did not explore its application to line-following robots. Additionally, some researchers have explored reinforcement learning to train robot control models; however, the training process is computationally intensive and typically requires high-performance hardware such as GPUs, making it costly and less accessible. This study addresses these gaps by proposing NEAT to evolve neural networks through genetic algorithms, autonomously optimizing line-tracking performance. NEAT eliminates manual PID tuning, adapts to dynamic environments (e.g., variable line widths), and inherently balances error minimization with path efficiency, offering a novel solution to the generalization challenges observed in traditional PID systems.

3 What is NEAT

NeuroEvolution of Augmenting Topologies (NEAT) is a genetic algorithm that offers a unique approach to artificial neural network (ANN) evolution, distinguishing itself by

simultaneously optimizing both the connection weights and the topology of the neural networks [12]. Unlike traditional neuroevolution methods that typically evolve only the weights of fixed-topology networks, NEAT begins with a population of simple, minimal networks and incrementally complexifies them through an evolutionary process. This incremental growth is facilitated by specific genetic operators, including mutations that add new nodes or connections, allowing the network to expand its structure as needed to solve the given task [12]. A crucial innovation in NEAT is the use of "historical markings" (innovation numbers) to track the origin of genes, which enables principled crossover between networks with disparate topologies, ensuring that functionally similar genes are aligned and recombined effectively [12]. Furthermore, NEAT employs a technique called "speciation," where individuals are grouped into different "species" based on their genetic similarity, preventing novel structural innovations from being outcompeted by existing well-adapted but simpler solutions early in the evolutionary process [12, 16]. To effectively differentiate between well-performing and less successful individuals, a comprehensive fitness function was essential. This function was designed to assign penalties for undesirable robot behaviors and rewards for preferred actions. The sum of these penalties and rewards constituted each genome's fitness score, enabling us to clearly distinguish between the most and least effective control policies. A more detailed explanation of this fitness function will be provided in the methodology section.
In the context of our line-following robot simulation, it was developed using PyGame, and NEAT was implemented as the robot's control system. The robots had 15 line sensors spread in a semi-circle shape. These sensors provided the inputs to the neural network. In the simulation, these inputs would be derived from the pixel data representing the line on the virtual track. The neural network's outputs, in turn, control the robot's motors, by using the two output nodes controlling the differential speed of the robot's left and right motors.

4 Generated Data

For the NEAT algorithm, a feedforward neural network (FNN) was developed using NEAT to follow a predetermined line path. The data was generated by running the evolved network on a robot in a simulation of a line following robot. To reduce bias, the simulation was repeated 10 times with the evolved network. Table 1 presents the features collected during the data collection process. Each row in the dataset represents the state of the robot at a given attempt and step in the simulation, and the action taken by the NEAT-based controller. The 'speed' feature serves as an indicator that shows at what max speed the NEAT controller was trained on; this helps in understanding at which speed a NEAT based controller would perform the best. The *attempt_id* serves as the trial number in which a given row was recorded, while the
To ensure a comparable evaluation, data for the PID-controlled line-following robot was collected using a similar methodology to that of the NEAT algorithm. The PID controller was re-implemented and optimized according to the method described in [7], following their original code. This optimized PID controller was then run on the

same track as NEAT for 10 trials, mirroring the NEAT evaluation process to minimize bias and account for potential outliers.

Table 1: Dataset features and their descriptions for generated data from NEAT and PID.

Feature	Description
attempt_id	The index of the robot's line-following attempt, incremented for each trial
step	An integer representing the number of update cycle in the simulation
sensor0 to sensor14	Line sensor readings from 15 onboard sensors [0 or 1]
action_L	NEAT-computed motor speeds for the left motor [-1, 1]
action_R	NEAT-computed motor speeds for the right motor [-1, 1]

Table 1 presents the features collected during the data collection process. Each row in the dataset represents the state of the robot at a given attempt and step in the simulation, and the action taken by the NEAT-based controller. The 'attempt_id' serves as the trial number a given row was logged at, while 'step' gives the precise simulation step each row was collected at for time series analysis. Sensor inputs include fifteen-line sensor readings (sensor0 to sensor14), which were used to determine the robot's position relative to the line. Initially, data on the robot's angular velocity and orientation was collected to gain insight into its motion. However, this information proved redundant, as it did not offer novel insights beyond what was already discernible from the motor speeds and sensor readings. The 'action_L' and 'action_R' feature contains the motors speed computed using the NEAT-based motor controller, this data is crucial to understanding the robot's response to each given state for further refinement in the future.

To ensure a comparable evaluation, data for the PID-controlled line-following robot was collected using a similar methodology to that of the NEAT algorithm. The PID controller was re-implemented and optimized according to the method described in [7], following their original code. This optimized PID controller was then run on the same track as NEAT for 10 trials, mirroring the NEAT evaluation process to minimize bias and account for potential outliers.

5 Research Methodology

We developed a top-down view 2D simulation using the Python library PyGame [17], this simulation was used to test the PID controller and train the NEAT generated feed forward network using the NEAT-Python library [18]. As shown in Figure 1, we developed a differential-drive robot with 15-line sensors in semi-circle shape. The simulation had many parameters and configurations for the environment and the robot, this includes wheels spacing, max speed, acceleration, wheels friction, motor no load rpm, and motor stall torque. The robustness of the simulation was first checked by testing the PID controller in it and fine tuning the PID constants until the robot was able to stay consistently on the line.

The simulated track is designed with multiple features to enhance the evaluation of robot performance. While the primary path is defined by a standard black line that the robot must follow, additional colored regions are integrated to support the fitness function, as shown in Figure 1. A red area signifies a restricted zone; if the robot enters it, the associated genome is penalized and terminated for that generation. Yellow zones represent bonus regions, rewarding genomes that exhibit desirable behaviors. Finally, green areas mark the successful completion of the track, granting a fitness reward and concluding the genome's evaluation.

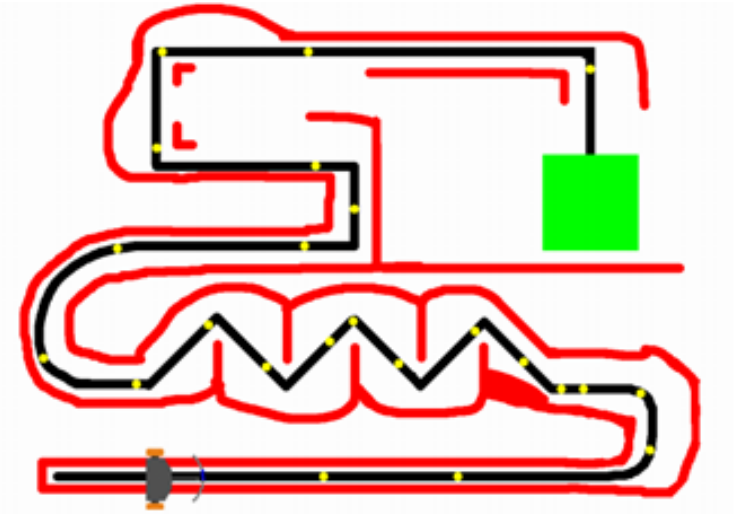


Fig. 1: The track of the simulation of line following robot with different color for evaluation

5.1 NeuroEvolution of Augmenting Topologies (NEAT)

NeuroEvolution of Augmenting Topologies (NEAT) is the test subject of this research; NEAT was implemented using the NEAT-Python library. Each genome encoded an Artificial Neural Network (ANN) mapping 15 sensor inputs to two motor speed outputs. NEAT works by optimizing the ANN's nodes, connections, weights, and biases to maximize the fitness calculated by the fitness function.

The fitness F for each robot is computed as the cumulative sum of rewards and penalties over each time step t until termination T, which occurs upon encountering a green or red zone or the simulation's end. The mathematical formulation of the fitness function is given in Equation 1, which contains the sum of reward or punishment functions. In Equation 2, the speed-related reward R_{speed} is calculated at time t based on the current linear velocity $v(t)$, the angular velocity $\omega(t)$, and a condition $CS(t)$, where $CS(t) = \sum_{j \in C} s_j(t)$ is the sum of the three center sensors C, with $s_j(t)$ being

the sensor reading at index j. A middle sensor bonus B_{middle}, as shown in Equation 3, is added if the middle line sensor detects the existence of the line beneath it.

Additionally, to encourage center tendency while following the line, Equation 4 was developed to give a reward for detecting the line under the three middle sensors only when the velocity of the robot is higher than 3. This condition was added because it was observed during the training process that each genome had a tendency to fully stop the robot when any of the three middle sensors detected the line, which led to several issues during the training process. The modification of the center tendency equation ensured that the robot moved while detecting the line.

To ensure smooth line tracking with no jittering, Equation 5 was added, encouraging stable motion by penalizing large differences between left and right motor speeds. Moreover, it was observed that some robots spun in place at high speeds while detecting the line briefly, then leaving and re-detecting it due to oscillatory movement. To address this, a steering penalty was introduced in Equation 6 to penalize high angular velocities.

When the robot leaves the track and no line is detected, a penalty function P_{off} is applied, described in Equation 7, where $\tau(t)$ is the consecutive off-track duration at t. To encourage high-speed movement and punish slow movement, a speed penalty function P_{speed}, as shown in Equation 8, is applied. This function returns a negative fitness if the robot moves slower than the minimum speed ('min_speed'), defined as half of the maximum speed allowed during training after conversion to m/s.

Specific areas with distinct colors on the track were added to encourage or discourage certain behaviors: red indicates a penalty, green means track completion (reward), and yellow provides an intermediate reward for completing a section of the track. Equations 9–11 define these functions, using an indicator function $\delta(\text{cond})$ that returns 1 if the condition holds, and 0 otherwise. To guide and influence individual's behavior, specific-colored regions were incorporated into the track, each representing a distinct objective or feedback mechanism. For example, red regions indicate undesirable behavior (penalty), green denotes successful track completion (reward), and yellow provides intermediate rewards for reaching key milestones. To integrate these regions into the fitness evaluation, Equations 9 , 10 , and 11 were formulated. Each equation assigns a corresponding reward or penalty based on the color of the area traversed, using the indicator function , which returns 1 if the condition holds and 0 otherwise.

$$F = \sum_{t=0}^{T} \Big(R_{\text{speed}}(t) + B_{\text{middle}}(t) + B_{\text{center}}(t) + B_{\text{smooth}}(t) - P_{\text{off}}(t) - P_{\text{speed}}(t) - P_{\text{steer}}(t) + B_{\text{yellow}}(t) \Big) + B_{\text{green}}(T) - P_{\text{red}}(T) \tag{1}$$

$$R_{\text{speed}}(t) = \begin{cases} 5 \cdot |v(t)|, & \text{if } CS(t) \geq 2 \text{ and } |\omega(t)| > 20^\circ/\text{step} \\ 10.0 \cdot |v(t)|, & \text{if } CS(t) \geq 2 \text{ and } |\omega(t)| \leq 20^\circ/\text{step} \\ 0.25 \cdot |v(t)|, & \text{if } CS(t) < 2 \text{ and } |\omega(t)| > 20^\circ/\text{step} \\ 0.5 \cdot |v(t)|, & \text{otherwise} \end{cases} \tag{2}$$

$$B_{\text{middle}}(t) = 50 \cdot \delta(s_{\text{mid}}(t)), \quad \delta(x) = \begin{cases} 1, & \text{if } x = 1 \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

$$B_{\text{center}}(t) = \begin{cases} 2 \cdot CS(t)^{1.5}, & \text{if } v(t) > 3 \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

$$B_{\text{smooth}}(t) = 0.5 \cdot \max\left(0,\ 2 - |v_{\text{left}}(t) - v_{\text{right}}(t)|\right) \tag{5}$$

$$P_{\text{steer}}(t) = \min\left(0.8 \cdot |\omega(t)|,\ 30\right) \tag{6}$$

$$P_{\text{off}}(t) = \begin{cases} 50 + \tau_t \cdot 50, & \text{if no line detected} \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

$$P_{\text{speed}}(t) = \begin{cases} 1000 \cdot |v(t)| + 0.05, & \text{if } v(t) \leq 0 \\ 100 \cdot |v(t)| - \text{min_speed}, & \text{if } 0 < v(t) < \text{min_speed} \\ 100 \cdot \text{min_speed} - v(t), & \text{if } v(t) \geq \text{min_speed} \end{cases} \tag{8}$$

$$B_{\text{yellow}}(t) = 500 \cdot \delta(\text{yellow zone detected at } t) \tag{9}$$

$$B_{\text{green}}(t) = \left(5000 + 100 \cdot |v(t)|\right) \cdot \delta(\text{green zone at } t) \tag{10}$$

$$P_{\text{red}}(t) = 500 \cdot \delta(\text{red zone at } t) \tag{11}$$

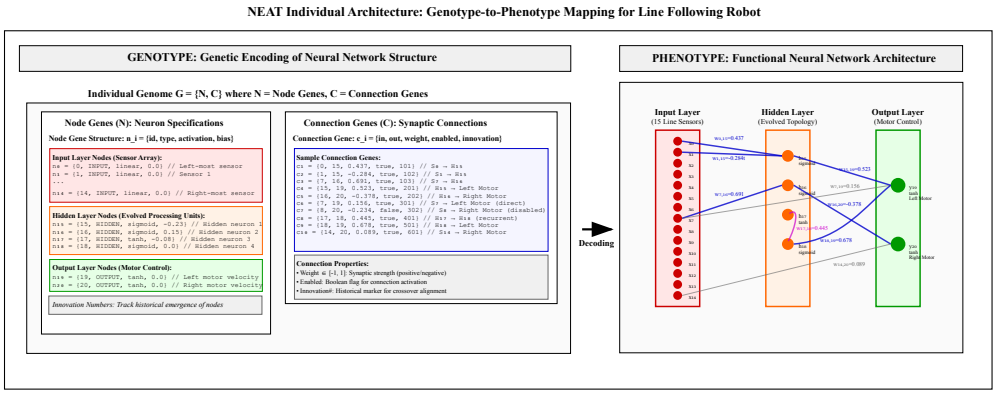


Fig. 2: Representation of Neural Network Structure in NEAT: (Left) The Genotype illustrates the genetic encoding of node and connection genes. (Right) The Phenotype depicts the functional neural network architecture.

Figure 2 illustrates the fundamental principles of the NEAT algorithm's representation of an individual neural network, encompassing both its genotype and phenotype. On the left, the Genotype: Genetic Encoding of Neural Network Structure section

details the evolutionary building blocks. This includes Node Genes, specifying unique identifiers, activation functions (e.g., sigmoid, tanh, linear), and bias values for each neuron, categorized into Input Layer (15-line sensors), Hidden Layer (evolved processing units), and Output Layer (motor control) nodes. Correspondingly, Connection Genes define the synaptic connections between these nodes, characterized by their input and output node IDs, assigned weight, an enablement flag (indicating whether the connection is active), and an innovation number. This innovation number is crucial for tracking the historical origin of genes and for facilitating proper crossover during evolution. On the right, the Phenotype: Functional Neural Network Architecture visually translates this genetic encoding into an operational neural network. This depicts the evolved topology, showing the 15 input sensors feeding into a dynamically structured hidden layer, which then connects to the two output nodes responsible for motor control (left and right wheel velocities). The blue lines represent active connections with their respective weights, forming the functional architecture that directly dictates the robot's behavior. This clear distinction between the genetic blueprint and its functional manifestation is central to NEAT's ability to evolve complex and efficient network topologies.

Our initial NEAT training attempt involved setting the robot's maximum speed to its highest possible value from the outset. However, under these conditions, the NEAT algorithm did not converge to a robust solution capable of fast line following. Consequently, the maximum speed was reduced, and the training process was reinitiated for 400 generations. Following this training, the individual with the highest fitness score was selected for a comprehensive evaluation. This best-performing individual was then tested across multiple trials to assess its line-tracking performance and lap time. During these evaluation trials, the simulation was initialized, and the robot's sensory inputs were processed by the evolved ANN. The ANN's outputs commanded the left and right motors, guiding the robot along the track until it reached the designated endpoint. Data collected during these trials allowed for the analysis of the NEAT-based controller's performance, including the computation of error metrics such as the Root Mean Square Error (RMSE) of line deviation.

Following the initial findings, a second training approach was implemented for the NEAT controller. This involved training the NEAT algorithm independently at nine distinct speeds, ranging from 100 to 500 pixels/second in increments of 50, for a duration of 500 generations per speed. This strategy was designed to allow each operational speed to undergo a dedicated evolutionary process, allowing the NEAT controller to optimally adapt to its respective speed. The primary objective of this approach was to identify the specific speed at which the NEAT controller would achieve optimal performance.

Figure 3 illustrates the comprehensive methodology used to train the line-following robot using the NEAT algorithm. Training begins with an outer iterative loop that systematically explores a predetermined range of 'MAX_SPEED' values, specifically from 100 to 500 with a 50 increment each time. For each designated speed, the system proceeds to load the necessary NEAT and robot configurations and initializes or restores the NEAT population. An inner loop then begins, iterating through multiple generations (up to 500) to evolve the robot's control policy at the current

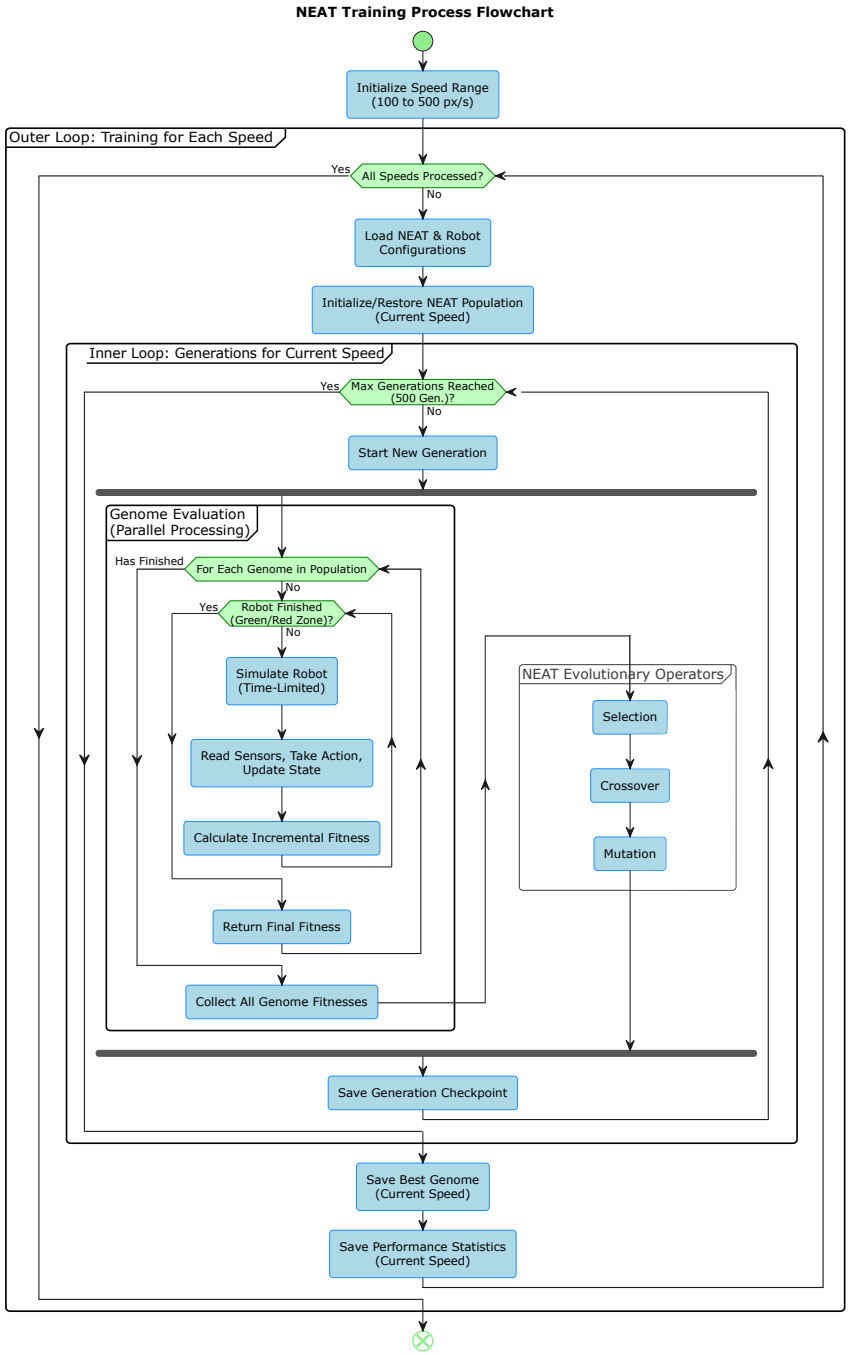


Fig. 3: NEAT Training Process Flowchart

'MAX_SPEED '. Each generation begins by preparing for the evaluation of individual genomes, utilizing parallel processing on all CPU cores to speed up the training process in headless mode. Within the parallel genome evaluation, each robot instance independently executes a simulation, reading line sensor data, taking actions based on its neural network's output, and incrementally calculating its fitness. This process continues until the robot reaches a defined "Green/Red Zone" or its time limit expires, at which point its final fitness is returned. After all genomes in a generation have been evaluated and their fitness scores collected, the NEAT evolutionary operators - Selection, Crossover, and Mutation - are applied to generate the next generation of genomes. A checkpoint is saved, and the inner loop continues until the maximum number of generations is reached. Once generations for a specific 'MAX_SPEED' are complete, the best-performing genome and relevant performance statistics for that speed are saved, and the outer loop progresses to the next 'MAX_SPEED' until all specified speeds have been explored.

5.2 PID

For comparison with the NEAT-based controller, a discrete-time PID controller was implemented following the formulation in [7]. At each control step k, the corrective signal $u[k]$ is updated according to Equation (12), where K_p, K_i, and K_d are the proportional, integral, and derivative gains (tuned heuristically to 2, 0.0001, and 0.5, respectively), and $e[k]$ is the line-position error at step k. The error itself is computed as a normalized, sensor-weighted average as shown in Equation (13), where n is the number of sensors (i.e., 15) and i is the index of the sensor, if the line is present. If the line is absent from all the sensors, the robot only mimics the last error. Finally, $u[k]$ is applied to the differential drive about a base speed $v_0 = 0.8$ of the max speed of the robot, with wheel velocities applied in Equation (14).

$$u[k] = u[k-1] + K_p\big(e[k] - e[k-1]\big) + K_i e[k] + K_d\big(e[k] - 2e[k-1] + e[k-2]\big) \quad (12)$$

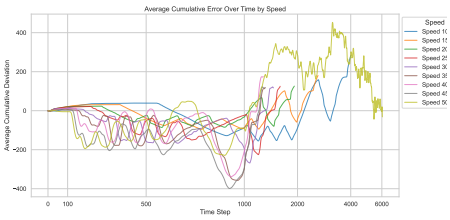
$$e[k] = \frac{\sum_{i=0}^{n} w_i \cdot \delta_i[k]}{\max\left(1, \sum_{i=0}^{n} \delta_i[k]\right)}, \quad w_i = \text{linspace}\left(-\frac{n}{2}, \frac{n}{2}, n\right), \quad \delta_i[k] = \begin{cases} 1, & \text{if sensor } i \text{ activated,} \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

$$v_{\text{left}}[k] = \text{clip}(v_0 - u[k], -1, 1), \quad v_{\text{right}}[k] = \text{clip}(v_0 + u[k], -1, 1) \quad (14)$$

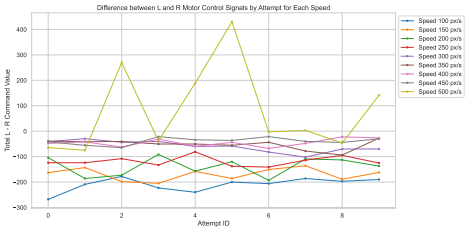
6 Results and Discussion

6.1 PID

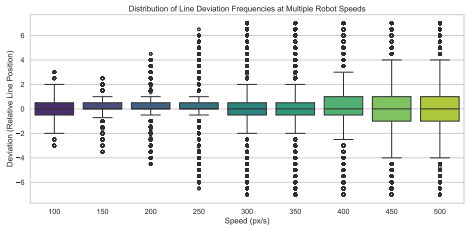
This section discusses the results of the analysis of the collected data either by implementing the PID controller or the NEAT-based controller.



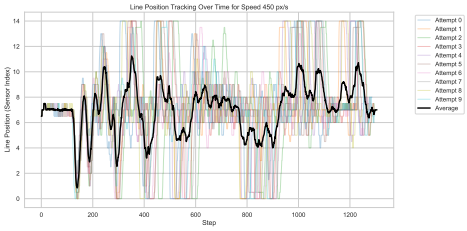
(a) Average cumulative error over time by speed.



(b) Difference between left and right motor control signals by attempt for each speed.



(c) Distribution of instantaneous deviation values at each speed.



(d) Line position relative to the sensor array over time for a speed of 450 pixels/s.

Fig. 4: Evaluation of the robot performance using PID.

In this section, we evaluate the performance of the PID controller across different speeds, each with 10 attempts to remove any run-to-run variability and ensure statistical reliability. A key metric in our analysis is the cumulative error (i.e., the deviation of the line from the sensor center). For each speed, error metrics, such as total error and Root Mean Squared Error (RMSE), were calculated to understand the performance of the PID controller.

In Figure 4a , we can see the cumulative error of the line position relative to the line sensor. From this chart, it is clear that the cumulative deviation changes significantly at the beginning due to the zig-zag section of the track, where the robot keeps changing its direction because of the track design. Despite this, it still manages to recover and maintain alignment for most speeds, except at 500 pixels/s. At this speed, the robot only completed the lap 5 out of 10 times due to overshooting most 90° turns, reversing, and moving in the wrong direction.

To quantify tracking performance, we computed the cumulative deviation of the line position relative to the center of the sensor array over time. The robot was equipped with 15 digital line sensors arranged in a semi-circular pattern, indexed from 0 to 14. At each time step, the estimated line position $p^{(a,t)}$ was determined from the active sensors. To center the deviation around the robot's midpoint, we subtracted 7 from this result, corresponding to the index of the middle sensor. A deviation of 0 indicates perfect alignment, while negative and positive values reflect leftward and rightward offsets, respectively. The cumulative error was computed as the sum of the

deviation at each time step for a trial. For each speed, cumulative errors were averaged over 10 independent attempts to account for run-to-run variability. All data were sampled at 60 Hz.

Figure 4c illustrates the distribution of line deviation frequencies, sampled at 60 Hz and aggregated across 10 trials for each speed. The y-axis represents the deviation of the robot's perceived center from the actual line, where negative values denote leftward veering and positive values denote rightward veering. At lower speeds (100–350 pixels/s), box plots show tight clustering of deviations around 0, with narrow interquartile ranges and minimal outliers, indicating that the PID controller maintained alignment with high consistency. However, speeds above 350 pixels/s show broader distributions, particularly at 450 and 500 pixels/s, where the interquartile ranges expand and more extreme deviations occur. This indicates a degradation in precision and robustness at higher speeds.

Figure 4d shows the line position over time at a speed of 450 pixels/s, where the PID controller achieved the fastest lap time of 20.433 s (Table 2). The plot illustrates how the robot deviates from the track at sharp turns but manages to realign with the center line. The line position $p^{(a,t)} \in [0, 14]$ at attempt a and time step t is computed as in (15), where $s_i^{(a,t)} > 0$ indicates that sensor i is active:

$$p^{(a,t)} = \begin{cases} \dfrac{\sum_{i=0}^{14} i \cdot s_i^{(a,t)}}{\sum_{i=0}^{14} s_i^{(a,t)}}, & \text{if } \sum_{i=0}^{14} s_i^{(a,t)} > 0, \\ p^{(a,t-1)}, & \text{otherwise.} \end{cases} \tag{15}$$

Edge cases:

- Leftmost sensors active: $p^{(a,t)} = 0$
- Rightmost sensors active: $p^{(a,t)} = 14$
- Single sensor active: $p^{(a,t)} =$ sensor index

Figure 4b shows the speed difference between left and right motors (left−right). This figure shows how well the PID controller managed to keep the difference between the left and right motors' speed balanced, especially at the speeds 400 and 450. Conversely at the high speed of 500 pixels/second the difference was pronounced, the controller generally at that speed preferred the left motor over the right motor, hence the positive difference, this could be because of the consistent overshooting the robot does because of the high speed.

Table 2 summarizes the total error cumulated, $RMSE$, average lap time, and the minimum (i.e., shortest) time for lap completion over 10 attempts for each speed, plus minus their standard deviation. The total error, which is the cumulative sum of the line deviation for each attempt, was surprisingly the highest at the lowest speed of 100 pixels/second with an average total error of 277.492, this positive error also suggests the possible tendency to have the line shifted a little bit to the right side of the robot. The speed of 450 pixels/second achieved the lowest absolute total error with a relatively high $RMSE$ (3.165), it also finished the lap with a time of 20.433 making it the fastest lap time by the PID controller. The slowest lap time was at speed 100,

Speed	Avg Total Commulative Error	Avg RMSE	Avg Time (s)	Min Time (s)
100	277.492 (±0.541)	0.546 (±0.381)	66.455 (±0.175)	66.217
150	169.208 (±0.600)	0.604 (±0.442)	42.998 (±0.214)	42.800
200	123.987 (±0.851)	0.852 (±0.685)	31.865 (±0.071)	31.750
250	117.112 (±1.599)	1.599 (±1.340)	26.277 (±0.240)	25.850
300	119.284 (±2.183)	2.184 (±1.810)	23.953 (±0.238)	23.633
350	109.271 (±2.748)	2.748 (±2.228)	21.440 (±0.310)	20.867
400	171.900 (±3.045)	3.047 (±2.400)	20.973 (±0.246)	20.550
450	106.864 (±3.167)	3.165 (±2.400)	21.300 (±0.452)	20.433
500	152.081 (±3.173)	3.214 (±2.367)	68.818 (±35.415)	21.867

Table 2: PID Attempts Error Metrics at Different Speeds each for 10 Attempts

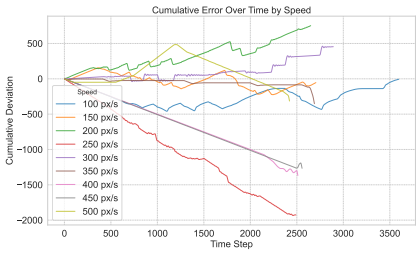
which is expected with such a low speed. Analysis of the PID controller's performance across varying speeds revealed a consistent trend: a marked increase in the standard deviation of average task completion time, Root Mean Squared Error ($RMSE$), and total error. This escalating variability signifies a substantial decrease in the controller's predictability and consistency at higher operational speeds. These findings collectively indicate that the fixed set of proportional (K_p), integral (K_i), and derivative (K_d) constants, while effective at lower speeds, can only facilitate an optimal balance between robot speed and control stability up to a certain performance threshold. Beyond this point, the observed degradation in performance strongly suggests the necessity for recalibration or an adaptive tuning mechanism for the PID parameters.

6.2 NEAT Analysis

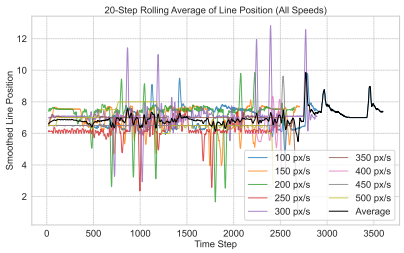
For the NEAT algorithm, training was conducted at various maximum robot speeds over 500 generations. As shown in Figure 5e, the evolution of best fitness across generations for each speed reveals a clear performance trend. Speeds from 100 to 300 px/s successfully learned to navigate the path and maintain a centered position, achieving high fitness values that plateaued quickly. In contrast, speeds ranging from 350 to 500 px/s consistently failed to learn correct steering control, with the robots often losing the line after a brief period of traversal. This indicates that the NEAT controller struggled to achieve stability and path completion at higher velocities.

Figure 5a presents the cumulative error, which measures the robot's deviation from the central line sensor. A cumulative error of -31.60 at 150 px/s, which was the lowest recorded, demonstrates strong performance at this speed. Another notable result was the cumulative error of -48.40 at 350 px/s. The negative sign of these errors suggests a tendency for the robot to maintain the line on the left side of the sensor's central position. However, the seemingly low error of 98.97 for the 500 px/s speed is misleading. The robot at this speed drifted, corrected itself but in the wrong direction, and then veered off the track completely, leading to an incomplete path and a low final error that does not reflect stable performance. This highlights the algorithm's inability to consistently and correctly steer the robot at high speeds.

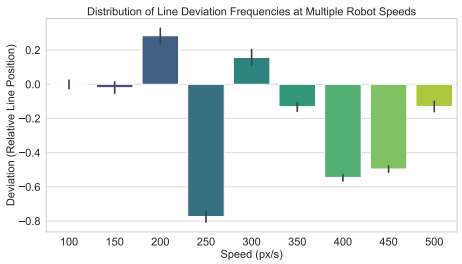
Analysis of the higher speed trials suggests a common behavioral pattern where the algorithm, unable to maintain stability, defaults to a strategy of slowing down to avoid crashes. This behavior indicates that for these high-speed scenarios, the optimal



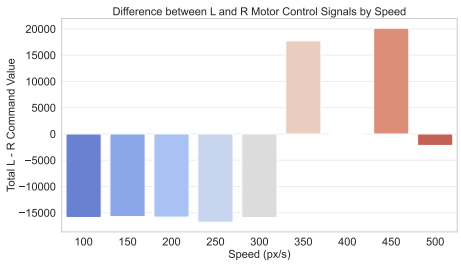
(a) Cumulative error over time by speed.



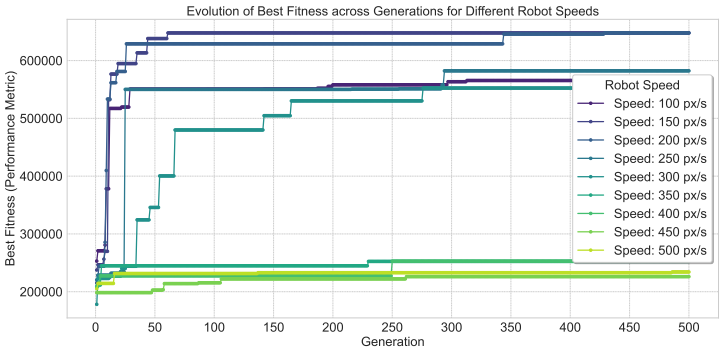
(b) Line position rolling average.



(c) Distribution of relative line deviation frequencies at multiple robot speeds.



(d) Difference between left and right motor control signals by speed.



(e) NEAT fitness evolution.

Fig. 5: Experimental Results and Performance Metrics.

solution found by the algorithm was to prioritize not crashing over path completion, a fundamental limitation in its learned control policy.

Furthermore, lower speeds (100-300 px/s) demonstrated a tendency to stop at sharp turns before adjusting their direction, a behavior that differs from a well-tuned PID controller, which would navigate these turns smoothly.

A closer look at the 350 px/s run reveals that the robot started slowly, then accelerated at the first turn and hit the red line. For the 400 px/s run, the robot was slow initially but then suddenly increased its speed mid-path, crashing on the rightmost red line. These erratic behaviors at higher speeds are reflected in the spikes seen in the rolling average line position (Figure 5b).

Figure 5c shows the relative line position, calculated using the formula $p_{rel}^{(a,t)} = p^{(a,t)} - 7$, recall Equation (15) to find the line position, which corresponds to a deviation from the middle sensor (sensor index 7). The data shows that for speeds from 250 px/s and above, with the exception of 300 px/s, the cumulative deviation is consistently negative. This implies that at these higher speeds, the algorithm learned to balance the robot with the line on the left side of it.

The motor control signals provide further insight into the steering behavior. As shown in Figure 5d, at lower speeds, the left motor exhibits a dominant command value, contributing more to steering corrections. However, at higher speeds, particularly at 350 px/s and 450 px/s, the right motor command value becomes dominant, significantly influencing the steering direction. This change in control policy at high speeds likely contributes to the observed instability and eventual path failure.

Speed	Total Commulative Error	RMSE	Time (s)
100	−6.000	0.757	59.933
150	−53.786	0.905	45.083
200	752.500	1.216	44.150
250	−1926.544	1.165	41.500
300	454.000	1.286	48.200
350	−352.000	0.688	44.817
400	−1368.500	0.747	41.850
450	−1266.500	0.705	42.600
500	−314.500	0.778	40.350

Table 3: NEAT at Different Speeds Error Metrics

Table 3 presents the performance metrics of the NEAT controller at various speeds. The data indicates that the controller's ability to successfully navigate the track is highly dependent on velocity. The fastest completed lap time was 48.20 seconds, achieved at a speed of 300 px/s. While data points for higher speeds are present in the table, the robot consistently veered off the track and failed to complete the lap at speeds above 300 px/s. Additionally, the slowest successful lap time was 59.933 seconds at a speed of 100 px/s.

The NEAT algorithm's precision, as measured by the Root Mean Squared Error (RMSE), shows a significant relationship with speed. The RMSE represents the average deviation of the robot from the center of the line. At a speed of 300 px/s, the algorithm achieved an exceptionally low average RMSE of 0.916, demonstrating its superior ability to keep the robot centered on the line. However, the data also shows a peak RMSE of 1.286 at a speed of 300 px/s. This fluctuation suggests that while the algorithm is capable of high precision, increased velocity introduces more dynamic

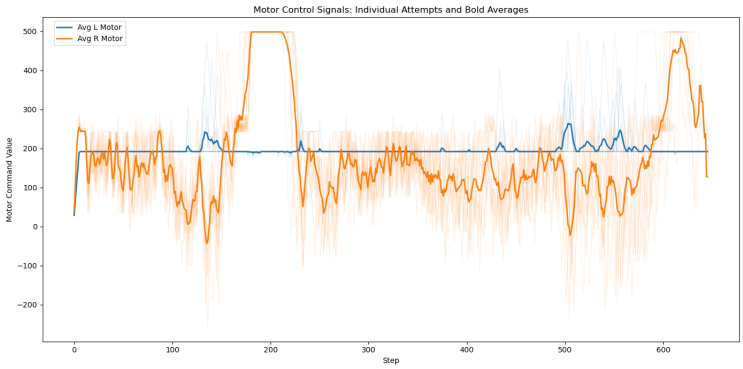


Fig. 6: Observed Asymmetry in Motor Control Commands from a Gradually Increasing Speed Training Methodology

challenges that can compromise a stable and consistent trajectory, likely contributing to the robot's inability to complete the track at speeds above 300 px/s.

6.2.1 Insights from the Gradually Increasing Speed Methodology

An alternative training methodology was explored, where the maximum robot speed was gradually increased across generations. While this approach did not result in a controller as robust as the one trained at a fixed high speed, it revealed a peculiar and noteworthy control strategy adopted by the NEAT algorithm.

A key observation from this training method was the evolution of an asymmetrical control policy, where the robot relied almost exclusively on a single motor for steering. As shown in Figure 6, the speed of one motor remained nearly constant while all steering corrections were executed by modulating the speed of the other motor. This is an interesting discovery because it shows that a functional control strategy was discovered by the evolutionary algorithm, even if it was not an optimal one. This behavior highlights the ability of NEAT to find unique, albeit unconventional, solutions to the control problem.

This learned asymmetry in motor control suggests a lack of generalizability, as this strategy would be less effective for diverse paths with varied turns. It also confirms the tendency of the evolutionary algorithm to optimize specifically for the presented track without developing a more generalized symmetrical control policy. The fact that the controller could complete the path using this method demonstrates NEAT's adaptive nature in exploiting specific environmental biases, but at the cost of robust, generalized performance.

7 Comparison of NEAT and PID

Based on the empirical results of this study, a direct comparison between the PID and NEAT algorithms reveals a critical trade-off between speed and precision in the context of line-following robots. While PID demonstrated superior performance in terms of lap time, the NEAT controller excelled in maintaining a precise trajectory.

The PID algorithm consistently completed the track at significantly higher speeds, with the robot avegaring a lap time of 20.93 second at velocities between 350 and 500 px/s. In contrast, the NEAT controller struggled to complete the full lap at these high speeds. It only succeeded at a maximum velocity of 300 px/s, with an average lap time of 48.20 seconds. Attempts to run the NEAT model at faster speeds resulted in crashes and an inability to finish the track. This finding suggests that for applications where high speed is the primary objective, a well-tuned PID controller is a more effective solution.

Conversely, the NEAT algorithm proved to be far more precise in its line following. It achieved a remarkably low average Root Mean Squared Error (RMSE) of 0.916, which is less than half the average RMSE of the PID controller (1.99). This data demonstrates the NEAT controller's superior ability to keep the robot centered on the line. Additionally, the final trained NEAT model showed high reproducibility, consistently producing the same result across multiple simulation runs at the same speed. This characteristic, which PID struggled to replicate, makes NEAT a potentially valuable solution for industrial applications such as warehouse robots that follow a predetermined path repeatedly.

However, a key limitation of the NEAT controller identified in this study is the learned asymmetrical motor control strategy, which suggests a lack of generalizability. Future work must address this to encourage more symmetrical control policies.

8 Conclusion

Based on our findings, we can conclude that the NEAT-based control system is not a viable and effective alternative to traditional PID controllers in scenarios where speed is the primary objective. While the NEAT controller demonstrated its adaptability and achieved a significantly lower Root Mean Squared Error in line deviation compared to PID, its performance at high speeds was insufficient. This indicates a clear trade-off: PID is the better choice for high-speed systems where lap time is crucial, whereas NEAT excels in applications where trajectory precision and reproducibility are necessary, such as in industrial warehouses.

The insights gained from this study also pave the way for several avenues of future research. While the NEAT controller demonstrated its adaptability, the learned asymmetrical motor control strategy suggests a lack of generalizability. Future work should explore methods to encourage more generalized and symmetrical control policies, perhaps through modifications to the fitness function or training environment. For higher-fidelity validation, migrating scenarios to ROS/Gazebo would provide standard sensor/actuator models and reproducible physics [19, 20]. This enables apples-to-apples controller benchmarking under identical world files. Additionally, to fully validate the efficiency and real-world applicability of the NEAT controller, it is crucial to move beyond purely simulated environments. Future research should prioritize integrating real-world testing on physical line-following robots and trying the algorithm on different track layouts and materials to identify and mitigate differences between simulated and real-world dynamics, as well as test the robustness of the evolved neural network.

References

[1] Saadatmand, S., Azizi, S., Kavousi, M. & Wunsch, D. Chakrabarti, S. & Paul, R. (eds) *Autonomous control of a line follower robot using a q-learning controller.* (eds Chakrabarti, S. & Paul, R.) *2020 10th Annual Computing and Communication Workshop and Conference (CCWC)*, 556–561 (2020).

[2] Tang, C. *et al.* Deep reinforcement learning for robotics: A survey of real-world successes. *arXiv.org* (2024). URL https://arxiv.org/abs/2408.03539.

[3] Darii, A., Nistor, M. S. & Pickl, S. Neuroevolution and neat at evolving neurocontrollers for obstacle avoidance for a robot arm (2024). URL https://ibn.idsi.md/sites/default/files/imag_file/Proceedings-IMCS60-IMI_2024-280-286.pdf. Accessed: Apr. 09, 2025.

[4] Sharma, A. & Kulhalli, S. G. – (ed.) *Warehouse automation using line follower robot.* (ed.–) *2023 4th IEEE Global Conference for Advancement in Technology (GCAT)*, 1–5 (IEEE, 2023).

[5] Lauckner, R. & Kolivand, H. Neat algorithm in autonomous vehicles. *Available at SSRN 4644203* (2023).

[6] Balaji, V., Balaji, M., Chandrasekaran, M., Khan, M. K. A. A. & Elamvazuthi, I. Optimization of pid control for high speed line tracking robots. *Procedia Computer Science* **76**, 147–154 (2015).

[7] Oguten, S. & Kabas, B. Pid controller optimization for low-cost line follower robots. *arXiv.org* (2021).

[8] Leal, H. M., Barbosa, R. S. & Jesus, I. S. Control of a mobile line-following robot using neural networks. *Algorithms* **18**, 51 (2025).

[9] Minaya Andino, C. A., Rosero, R., Zambrano, M. & Catota, P. Application of multilayer neural networks for controlling a line-following robot in robotic competitions. *Journal of Automation, Mobile Robotics and Intelligent Systems* 35–42 (2024).

[10] Kober, J., Bagnell, J. A. & Peters, J. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research* **32**, 1238–1274 (2013). URL https://doi.org/10.1177/0278364913495721.

[11] Arulkumaran, K., Deisenroth, M. P., Brundage, M. & Bharath, A. A. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine* **34**, 26–38 (2017).

[12] Stanley, K. O. & Miikkulainen, R. Evolving neural networks through augmenting topologies. *Evolutionary computation* **10**, 99–127 (2002).

[13] Stanley, K. O., Bryant, B. D. & Miikkulainen, R. Real-time neuroevolution in the nero video game. *Trans. Evol. Comp* **9**, 653–668 (2005). URL https://doi.org/10.1109/TEVC.2005.856210.

[14] Stanley, K. O., D'Ambrosio, D. B. & Gauci, J. A hypercube-based encoding for evolving large-scale neural networks. *Artificial life* **15**, 185–212 (2009).

[15] Alitappeh, R. J., Mahmoudi, N., Jafari, M. R. & Foladi, A. – (ed.) *Autonomous robot navigation: Deep learning approaches for line following and obstacle avoidance.* (ed.–) *2024 20th CSI International Symposium on Artificial Intelligence and Signal Processing (AISP)*, 1–6 (IEEE, 2024).

[16] Huang, J., Chen, Y., Mumtaz, J. & Zhong, L. Neuro-evolution of augmenting topologies for dynamic scheduling of flexible job shop problem. *Engineering Proceedings* **75** (2024).

[17] Pygame. Pygame (2019). URL https://www.pygame.org/news.

[18] Stanley, K. O. Welcome to neat-python's documentation! – neat-python 0.92 documentation (2015). URL https://neat-python.readthedocs.io/en/latest/. (accessed May 08, 2025).

[19] Macenski, S., Foote, T., Gerkey, B., Lalancette, C. & Woodall, W. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics* **7**, eabm6074 (2022).

[20] Koenig, N. & Howard, A. – (ed.) *Design and use paradigms for gazebo, an open-source multi-robot simulator.* (ed.–) *2004 IEEE/RSJ international conference on intelligent robots and systems (IROS)(IEEE Cat. No. 04CH37566)*, 2149–2154 (IEEE, 2004).